

ФГАОУ ВО «НИУ ИТМО»
ФАКУЛЬТЕТ ПРОГРАММНОЙ ИНЖЕНЕРИИ
И КОМПЬЮТЕРНОЙ ТЕХНИКИ

Отчёт №1

Введение в алгоритмы

(по дисциплине «Алгоритмы и структуры данных»)

Выполнил:

Лабин Макар Андреевич,
АиСД 3.2, Р3231, 466449.

Проверил:

Смирнов Виктор Игоревич



г. Санкт-Петербург, Россия
2026

Содержание

1	Яндекс.Контест	1
1.1	Агроном-любитель	1
1.1.1	Описание решения	1
1.1.2	Асимптотическая сложность	1
1.1.3	Листинг кода	1
1.2	Зоопарк Глеба	2
1.2.1	Описание решения	2
1.2.2	Асимптотическая сложность	2
1.2.3	Листинг кода	3
1.3	Конфигурационный файл	4
1.3.1	Описание решения	4
1.3.2	Асимптотическая сложность	4
1.3.3	Листинг кода	4
1.4	Профессор Хаос	5
1.4.1	Описание решения	5
1.4.2	Асимптотическая сложность	6
1.4.3	Листинг кода	6
2	Timus	7
2.1	Stone Pile за $\mathcal{O}(N \cdot 2^N)$	7
2.1.1	Описание решения	7
2.1.2	Асимптотическая сложность	7
2.1.3	Листинг кода	7
2.2	Stone Pile за $\mathcal{O}(N^2 \cdot W)$	8
2.2.1	Описание решения	8
2.2.2	Асимптотическая сложность	9
2.2.3	Листинг кода	9
2.3	Hyperjump за $\mathcal{O}(N^2)$	10
2.3.1	Описание решения	10
2.3.2	Асимптотическая сложность	10
2.3.3	Листинг кода	10
2.4	Hyperjump за $\mathcal{O}(N)$	11
2.4.1	Описание решения	11
2.4.2	Асимптотическая сложность	11
2.4.3	Листинг кода	11

1 Яндекс.Контест

1.1 Агроном-любитель

1.1.1 Описание решения

Переформулировав условие задачи, в массиве нужно найти наиболее длинный отрезок, в котором отсутствуют три одинаковых числа подряд.

Заметим, что для достижения максимальной длины отрезка необходимо, чтобы искомым отрезком начинался как можно раньше и заканчивался как можно позже. Другими словами, если i -ый отрезок можно расширить без нарушения условия — мы расширяем.

Рассмотрим ситуацию, когда в массиве в первый раз встретились три и более одинаковых числа подряд: $x_i = x_{i+1} = x_{i+2} = \dots$. Следуя наблюдению выше, первый отрезок должен сделать x_{i+1} своим последним элементом. Тогда следующий отрезок должен начинаться с x_{i+1} элемента. Для следующей встречи рассуждения аналогичны, как если бы исходный массив начинался с x_i элемента.

Учитывая, что отрезки пересекаются ровно по одному элементу, можно пройти по всему массиву один раз и вести учёт длины каждого отрезка, отдельно сохраняя наибольший из них.

1.1.2 Асимптотическая сложность

Временная сложность решения — линейная $\mathcal{O}(N)$. В алгоритме для каждого элемента массива выполняется константное количество операций.

Пространственная сложность решения — $\mathcal{O}(1)$. В решении при вычислении промежуточных значений используется константное количество памяти, исходный массив отдельно не сохраняется.

1.1.3 Листинг кода

```
1 #include <iostream>
2
3 struct segment {
4     int l;
5     int r;
6 };
7
8 int main() {
9     int N;
10    std::cin >> N;
11    struct {
12        short occur = 0;
13        long item = -1;
14    } history;
15    segment ans = {0, 0};
16    segment curr = {0, 0};
17    for (int i = 0; i < N; i++) {
18        long a;
```

```

19     std::cin >> a;
20     if (history.item != a) {
21         history.item = a;
22         history.occure = 1;
23     } else {
24         history.occure++;
25     }
26     if (history.occure == 3) {
27         if (ans.r - ans.l < curr.r - curr.l) {
28             ans = curr;
29         }
30         history.occure--;
31         curr.l = i - 1;
32     }
33     curr.r = i;
34 }
35 if (ans.r - ans.l < curr.r - curr.l) {
36     ans = curr;
37 }
38 std::cout << ans.l + 1 << " " << ans.r + 1;
39 return 0;
40 }

```

1.2 Зоопарк Глеба

1.2.1 Описание решения

По формулировке задача напоминает проверку модифицированной скобочной последовательности на корректность. Представим, что разные буквы обозначают разные виды скобок (*круглая, квадратная, фигурная и др.*), а животные с ловушками — тип скобки (*открытая/закрытая*). Только в задаче неважно, стоит ли сначала открытая скобка и затем закрытая или наоборот — оба варианта подойдут.

Чтобы проверить корректность последовательности, введём стек, который хранит структуры с метаданными элементов: его тип, вид и относительный индекс среди данного вида.

Запустим цикл по всему массиву и будем добавлять элементы в стек. Если текущий элемент одного вида с вершиной стека, но разного типа — удаляем вершину стека и переходим к следующему элементу.

Последовательность будет правильной, если стек после обхода массива будет пустой: значит, все животные попались в свои ловушки.

1.2.2 Асимптотическая сложность

Временная сложность решения — линейная $\mathcal{O}(N)$. В алгоритме используется обход по всему массиву, где для каждого элемента выполняется константное число операций.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. Для хранения N входных символов выделена строка в памяти, которую впоследствии обрабатывают при помощи стека и массива, каждый длины N в худшем случае.

1.2.3 Листинг кода

```
1 #include <iostream>
2 #include <stack>
3 #include <vector>
4
5 struct item {
6     char value;
7     bool isTrap;
8     int idx;
9 };
10
11 int main() {
12     std::string zoo;
13     std::cin >> zoo;
14     int N = zoo.length();
15     std::stack<item> stack;
16     std::vector<int> order;
17     for (int i = 0; i < N; i++) {
18         order.push_back(-1);
19     }
20     int animal_idx = 1, trap_idx = 1;
21     for (int i = 0; i < N; i++) {
22         bool isTrap = (char)std::tolower(zoo[i]) != zoo[i];
23         int idx = isTrap ? trap_idx++ : animal_idx++;
24         item curr = {(char)std::tolower(zoo[i]), isTrap, idx};
25         if (stack.size() == 0 || stack.top().value != curr.value ||
26             stack.top().isTrap == curr.isTrap) {
27             stack.push(curr);
28             continue;
29         }
30         item top = stack.top();
31         stack.pop();
32         if (top.isTrap) {
33             order[top.idx - 1] = curr.idx;
34         } else {
35             order[curr.idx - 1] = top.idx;
36         }
37     }
38     if (stack.size() > 0) {
39         std::cout << "Impossible";
40         return 0;
41     }
42     std::cout << "Possible" << std::endl;
43     for (int i = 0; i < N; i++) {
44         if (order[i] != -1) {
45             std::cout << order[i] << " ";
46         }
47     }
48     return 0;
49 }
```

1.3 Конфигурационный файл

1.3.1 Описание решения

Сфокусировавшись на потоке ввода, упростим задачу до трёх ситуаций:

1. Начало блока.
2. Конец блока.
3. Присваивание переменной значения.

Введём «глобальную» хеш-таблицу, которая хранит текущее состояние переменных. Ключи — имена переменных, значение по ключу — значение переменной.

Также введём стек, элементы которого будут эквивалентны вложенным блокам. Идея — хранить в нём историю изменений значений переменных. Вершина стека хранит значения переменных на прошлом уровне.

Когда из потока ввода поступает команда «начать новый блок», в стек будет записана пустая хеш-таблица. По мере обновления значений переменных в текущий блок будет записываться старое значение переменной, а в глобальную хеш-таблицу — актуальное значение.

Как только поступит команда «завершить текущий блок», из вершины стека удаляется блок. Затем, для каждой переменной из удаляемого блока мы восстанавливаем старое значение в глобальной хеш-таблице. Так мы восстанавливаем исходное состояние переменных до входа в очередной блок.

1.3.2 Асимптотическая сложность

Временная сложность решения — линейная $\mathcal{O}(N)$. Для каждой считанной строки выполняется константное количество операций в случае начала блока и обновления значения переменной. Если же блок завершается, то все предыдущие присвоения блока отменяются за константное время — в сумме такие операции не превысят линейное время работы.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. Для хранения текущего состояния используется стек, элементами которого являются хеш-таблицы. Так как и добавление элемента в стек, и добавление элемента в хеш-таблицу происходит по строчной команде, число всех элементов в сумме не превышает числа входных строк.

1.3.3 Листинг кода

```
1 #include <bits/stdc++.h>
2
3 #include <iostream>
4 #include <stack>
5 #include <unordered_map>
6
7 using scope = std::unordered_map<std::string, long>;
8
9 void update_scope(std::stack<scope>& ctx, scope& vars, std:::
    string lval, std::string rval) {
```

```

10 char* fail;
11 long rval_long = std::strtol(rval.c_str(), &fail, 10);
12 if (ctx.top().count(lval) == 0) {
13     ctx.top()[lval] = vars.count(lval) ? vars[lval] : 0;
14 }
15 if (*fail) {
16     vars[lval] = vars[rval];
17     std::cout << vars[lval] << "\n";
18 } else {
19     vars[lval] = rval_long;
20 }
21 }
22
23 int main() {
24     std::string line;
25     scope vars;
26     std::stack<scope> ctx;
27     ctx.push({});
28     while (std::cin >> line) {
29         if (line == "{") {
30             ctx.push({});
31         } else if (line == "}") {
32             // insert_range does not suit there:
33             // "if there is no element with key equivalent to the key
34             // of that element"
35             for (const auto& [key, value] : ctx.top()) {
36                 vars[key] = value;
37             }
38             ctx.pop();
39         } else {
40             std::stringstream ss(line);
41             std::string lval, rval;
42             std::getline(ss, lval, '=');
43             std::getline(ss, rval, '=');
44             update_scope(ctx, vars, lval, rval);
45         }
46     }
47     return 0;
48 }

```

1.4 Профессор Хаос

1.4.1 Описание решения

В задаче для числа дней эксперимента используется ограничение $1 \leq k \leq 10^{18}$, что намекает на математическую природу решения. Линейный алгоритм для моделирование задачи будет бессилён при больших k .

Опуская детали, на каждом дне эксперимента к количеству бактерий применяется линейная функция $f(x) = bx - c$. Заметим, что если $c = a \cdot (b - 1)$, то $f(a) = a$ в любой из дней эксперимента (a — *неподвижная точка* f).

Если после применения f число бактерий *увеличилось*, то на следующем шаге оно также будет увеличиваться в силу возрастания f . Но по условию задачи число бактерий ограничено d , поэтому при достижении этой отметки число бактерий останется неизменным в последующих днях эксперимента.

Если после применения f число бактерий *уменьшилось*, то на следующем шаге оно также будет уменьшаться в силу возрастания f . Значит, оно в какой-то из дней достигнет нуля.

1.4.2 Асимптотическая сложность

Временная сложность решения — $O(d)$. Если число бактерий не изменяется, ответ однозначен. Иначе будет выполнено не более d операций: функция целая, и она в худшем случае может расти или убывать на единицу до тех пор, пока не достигнет границ отрезка $[0, d]$.

Пространственная сложность решения — $O(1)$. В решении используются только переменные для хранения промежуточных данных и ввода/вывода.

1.4.3 Листинг кода

```
1 #include <iostream>
2
3 int main() {
4     int a, b, c, d;
5     long long k;
6     std::cin >> a >> b >> c >> d >> k;
7     if (c == a * (b - 1)) {
8         std::cout << a;
9         return 0;
10    }
11    long x = a;
12    for (int i = 0; i < (d < k ? d : k); i++) {
13        x *= b;
14        if (x < c) {
15            x = 0;
16            break;
17        }
18        x -= c;
19        if (x >= d) {
20            x = d;
21            break;
22        }
23    }
24    std::cout << x;
25    return 0;
26 }
```


2 Timus

2.1 Stone Pile за $\mathcal{O}(N \cdot 2^N)$

2.1.1 Описание решения

В задаче используется ограничение $0 \leq N \leq 20$, что намекает на использование алгоритма высокой сложности. Следовательно, можно попробовать перебрать всевозможные варианты неупорядоченных распределений элементов между двумя группами — их не более $2^{20} \approx 10^6$.

Для оптимизации использования памяти можно воспользоваться битовыми операциями. В операционных системах чаще всего выделяется 32 бит для `int`, поэтому он подойдёт для ограничений данной задачи.

Таким образом, на хранение вариантов распределения будет выделено хотя бы $2^{20} \cdot 2^5 \cdot 2^{-23} = 4$ МиБ, что допустимо для ограничений данной задачи.

Далее для каждого сгенерированного варианта при помощи битовых сдвигов вычисляем сумму элементов каждой группы. Если i -ый бит справа равен единице, то элемент принадлежит первой группе, иначе — второй.

После мы вычисляем модуль разности сумм и обновляем глобальный минимум в случае необходимости.

2.1.2 Асимптотическая сложность

Временная сложность решения — $\mathcal{O}(N \cdot 2^N)$. После считывания массива генерируются всевозможные варианты неупорядоченного распределения элементов по двум группам. Затем, для каждого варианта вычисляется искомая разность сумм двух групп за линейное время и обновляется глобальный минимум.

Пространственная сложность решения — $\mathcal{O}(2^N)$. Всевозможные варианты распределения камней между двумя кучами хранятся в массиве длины 2^N , исходные данные хранятся в массиве длины N .

2.1.3 Листинг кода

```
1 #include <iostream>
2 #include <vector>
3
4 void fill_options(std::vector<int>& options) {
5     for (int o : options) {
6         options.push_back(o << 1 | 1);
7         options.push_back(o << 1);
8     }
9 }
10
11 int main() {
12     int N;
13     std::cin >> N;
14     std::vector<int> arr;
15     for (int i = 0; i < N; i++) {
16         int item;
17         std::cin >> item;
```

```

18     arr.push_back(item);
19 }
20 std::vector<int> opts = {0, 1};
21 for (int i = 0; i < N - 1; i++) {
22     fill_options(opts);
23 }
24 long ans = 2000000;
25 for (int o : opts) {
26     long sum1 = 0, sum2 = 0, curr;
27     for (int i = 0; i < N; i++) {
28         if (o >> i & 1) {
29             sum1 += arr[i];
30         } else {
31             sum2 += arr[i];
32         }
33     }
34     curr = abs(sum2 - sum1);
35     if (curr < ans) {
36         ans = curr;
37     }
38 }
39 std::cout << ans;
40 return 0;
41 }

```

2.2 Stone Pile за $\mathcal{O}(N^2 \cdot W)$

2.2.1 Описание решения

Воспользуемся динамическим программированием для решения задачи, а именно приравняем её к задаче о рюкзаке. Введём следующие обозначения:

- $f(w, i)$ — минимальное число камней среди первых i камней в массиве, у которых масса в сумме хотя бы w .
- W — максимально возможная масса камня.
- W_{all} — сумма масс камней данного массива, $W_{\text{all}} \leq N \cdot W$.

Для базы рекуррентной формулы положим:

$$f(w, 0) = \begin{cases} 0, & \text{если } w = 0 \\ N + 1, & \text{если } w > 0 \end{cases}$$

Тогда шаг рекурренты выглядит следующим образом:

$$f(w, i) = \begin{cases} \min(f(w, i - 1), f(w - w_i, i - 1) + 1), & \text{если } w \geq w_i \\ f(w, i - 1), & \text{если } w < w_i \end{cases}$$

После вычисления значений функции, ответ будет во множестве:

$$X = \{f(w, N) \mid \forall w = \overline{0, W_{\text{all}}}, f(w, N) \neq N + 1\}$$

Ответ рассчитывается по формуле:

$$x = \min \{|2w - W_{\text{all}}| \mid f(w, N) \in X\}$$

Напоследок можно заметить, что рекуррентное выражение использует для подсчёта лишь предыдущий слой относительно индекса i . С этим наблюдением можно оптимизировать использование памяти и хранить вместо двумерного массива только два слоя: текущий и предыдущий.

2.2.2 Асимптотическая сложность

Временная сложность решения — $\mathcal{O}(N^2 \cdot W)$. Для расчёта ответа высчитывается $N \cdot W_{\text{all}} \leq N^2 \cdot W$ значений рекуррентного выражения. Ответ определяется на последнем шаге за линейное время.

Пространственная сложность решения — $\mathcal{O}(N + W)$. Для вычисления значений рекуррентного выражения используется два массива длины $W_{\text{all}} \leq W$, исходные данные хранятся в массиве длины N .

2.2.3 Листинг кода

```
1 #include <iostream>
2 #include <vector>
3
4 int main() {
5     int N;
6     std::cin >> N;
7     std::vector<int> arr;
8     long total_sum = 0;
9     for (int i = 0; i < N; i++) {
10         int item;
11         std::cin >> item;
12         arr.push_back(item);
13         total_sum += item;
14     }
15     std::vector<int> min_coins[2];
16     int INF = N + 1;
17     for (int j = 0; j < 2; j++) {
18         std::vector<int> step;
19         for (int i = 0; i <= total_sum; i++) {
20             step.push_back(INF);
21         }
22         step[0] = 0; // dp base
23         min_coins[j] = step;
24     }
25     short curr_step_idx = 0, prev_step_idx = 1;
26     for (int i = 1; i <= N; i++) {
27         curr_step_idx ^= 1;
28         prev_step_idx ^= 1;
29         for (long cost = 0; cost <= total_sum; cost++) {
30             int prev = min_coins[prev_step_idx][cost];
31             long prev_cost = cost - arr[i - 1];
```

```

32     int curr = prev_cost >= 0 ? min_coins[prev_step_idx][cost
    - arr[i - 1]] + 1 : INF;
33     min_coins[curr_step_idx][cost] = prev <= curr ? prev :
        curr;
34 }
35 }
36 long diff = total_sum;
37 for (long cost = 0; cost < total_sum; cost++) {
38     long curr_diff = abs(2 * cost - total_sum);
39     if (min_coins[curr_step_idx][cost] != INF && curr_diff <
        diff) {
40         diff = curr_diff;
41     }
42 }
43 std::cout << diff;
44 return 0;
45 }

```

2.3 Hyperjump за $\mathcal{O}(N^2)$

2.3.1 Описание решения

Переформулировав условие задачи, для данного массива нужно найти отрезок с максимальной суммой его элементов.

Учитывая ограничение $0 \leq N \leq 6 \cdot 10^4$, можно размышлять в сторону алгоритмов квадратичной сложности. Это означает, что мы можем перебрать всевозможные начала и концы отрезков. Но этого пока недостаточно для нахождения суммы элементов на конкретном отрезке.

Для решения такой задачи необходимо предварительно кешировать префиксные суммы элементов массива за линейное время. Это позволит оптимизировать расчёт суммы элементов отрезка с линейного времени до константного.

Остаётся фиксировать текущий максимум суммы на каждой итерации, чтобы получить глобальный максимум.

2.3.2 Асимптотическая сложность

Временная сложность решения — квадратичная $\mathcal{O}(N^2)$. В начале работы алгоритма мы за линейное время создаём массив префиксных сумм и заполняем его. Затем для каждой неупорядоченной пары элементов массива выполняем константное количество операций.

Пространственная сложность решения — линейная $\mathcal{O}(N)$. В решении используются два массива длины N и $N + 1$. При помощи них производятся вычисления, требующие константное количество памяти.

2.3.3 Листинг кода

```

1 #include <iostream>
2 #include <vector>
3

```

```

4 int main() {
5     int N;
6     std::cin >> N;
7     std::vector<int> arr;
8     std::vector<long long> prefix;
9     prefix.push_back(0);
10    for (int i = 0; i < N; i++) {
11        arr.push_back(0);
12        prefix.push_back(0);
13    }
14    for (int i = 0; i < N; i++) {
15        std::cin >> arr[i];
16        prefix[i + 1] = prefix[i] + arr[i];
17    }
18    long long ans = 0;
19    for (int i = 1; i <= N; i++) {
20        for (int j = i; j <= N; j++) {
21            long long curr = prefix[j] - prefix[i - 1];
22            if (curr > ans) {
23                ans = curr;
24            }
25        }
26    }
27    std::cout << ans;
28    return 0;
29 }

```

2.4 Hyperjump за $\mathcal{O}(N)$

2.4.1 Описание решения

Переформулировав условие задачи, для данного массива нужно найти отрезок с максимальной суммой его элементов.

Воспользуемся жадной стратегией: будем набирать элементы в отрезок до тех пор, пока их сумма неотрицательная. Иначе выгоднее начать новый отрезок: если продолжить набирать элементы в отрезок с отрицательной суммой, то мы будем в проигрыше на эту сумму.

Остаётся на каждой итерации фиксировать текущую сумму и обновлять глобальный максимум при необходимости.

2.4.2 Асимптотическая сложность

Временная сложность решения — линейная $\mathcal{O}(N)$. Для подсчёта требуемой суммы решение считывает весь массив поэлементно и выполняет для каждого из них константное число операций.

Пространственная сложность решения — константная $\mathcal{O}(1)$. Алгоритм использует для вычисления результата фиксированное число переменных, исходный массив в памяти не сохраняется.

2.4.3 Листинг кода

```
1 #include <iostream>
2
3 int main() {
4     int N;
5     std::cin >> N;
6     long long ans = 0, curr = 0;
7     for (int i = 0; i < N; i++) {
8         int item;
9         std::cin >> item;
10        curr += item;
11        if (curr < 0) {
12            curr = 0;
13        }
14        if (curr > ans) {
15            ans = curr;
16        }
17    }
18    std::cout << ans;
19    return 0;
20 }
```